CSA4704 - DEEP LEARNING
Lab Experiments

Name: Harini. R
Register no: 192324108

Experiment 1: Vector and Matrix Operations for Deep Learning
Problem Statement: An e-commerce firm represents customers and products as numeric vectors to power its recommendation engine. Every recommendation is a chain of dot products and matrix multiplications. A small online bookstore has 6 customers and 5 genres. Each customer has a rating vector, and each book has a genre-weight vector. The store wants to score every customer–book pair and find similar customers. The objective is to perform vector and matrix operations that underpin a neural network layer. Use NumPy, Matplotlib.
Tasks:
1. Build the customer–genre (6×5) and genre–book (5×4) matrices in NumPy.
2. Compute dot products, matrix product, transpose, inverse (where valid), and element-wise
operations.
3. Compute cosine similarity between customers and identify the two most similar.
4. Show that a dense layer is y = Wx + b by computing it manually for one customer.
5. Visualise the score matrix as a heatmap.

Formulas:

- Dot product: Measures the combined relationship between two vectors.
- Matrix multiplication: Combines customer preferences with book genre weights to calculate recommendation scores.
- Transpose: Interchanges rows and columns.
- Inverse: Reverses a square matrix when its determinant is nonzero.
- Cosine similarity: Measures the angle-based similarity between customer rating vectors.
- Dense layer: Calculates $y = Wx + b$, where W is the weight matrix and b is the bias vector.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics.pairwise import cosine_similarity

# 1. Customer-genre matrix (6 customers x 5 genres)
customer_genre = np.array([
```

```
    [5, 4, 1, 2, 1],
    [4, 5, 1, 1, 2],
    [1, 2, 5, 4, 1],
    [2, 1, 4, 5, 2],
    [5, 3, 2, 1, 1],
    [1, 1, 2, 3, 5]
], dtype=float)

# 2. Genre-book matrix (5 genres x 4 books)
genre_book = np.array([
    [0.9, 0.1, 0.2, 0.0],
    [0.8, 0.2, 0.1, 0.1],
    [0.1, 0.9, 0.2, 0.1],
    [0.2, 0.8, 0.3, 0.2],
    [0.1, 0.1, 0.8, 0.9]
])

print("Customer-Genre Matrix:\n", customer_genre)
print("\nGenre-Book Matrix:\n", genre_book)

# 3. Dot product of two customer vectors
dot = np.dot(customer_genre[0], customer_genre[1])
print("\nDot product of Customer 1 and Customer 2:", dot)

# 4. Matrix multiplication: recommendation score matrix
scores = np.dot(customer_genre, genre_book)
print("\nCustomer-Book Score Matrix:\n", scores)

# 5. Transpose
print("\nTranspose of Customer-Genre Matrix:\n",
      customer_genre.T)

# 6. Inverse, where valid
# A square Gram matrix is used because customer_genre is not square.
gram = customer_genre.T @ customer_genre
print("\nGram Matrix:\n", gram)
print("Gram matrix rank:", np.linalg.matrix_rank(gram))

# Add a small diagonal term to make a stable, invertible matrix.
regularized = gram + 0.01 * np.eye(gram.shape[0])
```

```
inverse_matrix = np.linalg.inv(regularized)
print("\nInverse of Regularized Gram Matrix:\n", inverse_matrix)

# 7. Element-wise operations
print("\nElement-wise addition:\n", customer_genre + 1)
print("\nElement-wise multiplication:\n",
      customer_genre * customer_genre)

# 8. Cosine similarity between customers
similarity = cosine_similarity(customer_genre)
print("\nCosine Similarity Matrix:\n", np.round(similarity, 3))

# Find the most similar pair without comparing a customer to itself
np.fill_diagonal(similarity, -1)
i, j = np.unravel_index(np.argmax(similarity), similarity.shape)
print(f"\nMost similar customers: {i+1} and {j+1}")
print("Cosine similarity:", round(similarity[i, j], 3))

# 9. Dense layer: y = Wx + b for Customer 1
x = customer_genre[0]        # 5 genre values
W = genre_book.T             # 4 books x 5 genres
b = np.array([0.5, 0.2, 0.1, 0.3])

y = W @ x + b
y_manual = np.array([
    sum(W[k, t] * x[t] for t in range(5)) + b[k]
    for k in range(4)
])

print("\nDense layer output:", y)
print("Manual calculation:", y_manual)
print("Matches:", np.allclose(y, y_manual))

# 10. Heatmap of customer-book scores
plt.figure(figsize=(8, 5))
plt.imshow(scores, cmap="YlGnBu", aspect="auto")
plt.colorbar(label="Recommendation Score")
plt.xticks(range(4), ["Book 1", "Book 2", "Book 3", "Book 4"])
plt.yticks(range(6), ["Customer 1", "Customer 2",
                      "Customer 3", "Customer 4",
```

```
                "Customer 5", "Customer 6"])
plt.title("Customer-Book Recommendation Scores")
plt.xlabel("Books")
plt.ylabel("Customers")

for r in range(scores.shape[0]):
    for c in range(scores.shape[1]):
        plt.text(c, r, f"{scores[r, c]:.1f}",
                 ha="center", va="center")

plt.tight_layout()
plt.show()
```

Output:

```
Customer-Genre Matrix:
 [[5. 4. 1. 2. 1.]
 [4. 5. 1. 1. 2.]
 [1. 2. 5. 4. 1.]
 [2. 1. 4. 5. 2.]
 [5. 3. 2. 1. 1.]
 [1. 1. 2. 3. 5.]]

Genre-Book Matrix:
 [[0.9 0.1 0.2 0. ]
 [0.8 0.2 0.1 0.1]
 [0.1 0.9 0.2 0.1]
 [0.2 0.8 0.3 0.2]
 [0.1 0.1 0.8 0.9]]

Dot product of Customer 1 and Customer 2: 45.0

Customer-Book Score Matrix:
 [[8.3 3.9 3.  1.8]
 [8.1 3.3 3.4 2.6]
 [3.9 8.3 3.4 2.4]
 [4.2 8.2 4.4 3.3]
 [7.4 3.8 2.8 1.6]
 [3.  5.  5.6 5.4]]

Transpose of Customer-Genre Matrix:
 [[5. 4. 1. 2. 5. 1.]
 [4. 5. 2. 1. 3. 1.]
 [1. 1. 5. 4. 2. 2.]
 [2. 1. 4. 5. 1. 3.]
 [1. 2. 1. 2. 1. 5.]]
```

```
Gram Matrix:
 [[72. 60. 34. 36. 28.]
 [60. 56. 31. 32. 26.]
 [34. 31. 51. 51. 28.]
 [36. 32. 51. 56. 34.]
 [28. 26. 28. 34. 36.]]
Gram matrix rank: 5

Inverse of Regularized Gram Matrix:
 [[ 0.13503052 -0.14523623  0.03242334 -0.04205744  0.01436786]
 [-0.14523623  0.18701046 -0.05894299  0.0603663  -0.03326043]
 [ 0.03242334 -0.05894299  0.27645087 -0.27862432  0.06546145]
 [-0.04205744  0.0603663  -0.27862432  0.32609462 -0.10212862]
 [ 0.01436786 -0.03326043  0.06546145 -0.10212862  0.08614062]]

Element-wise addition:
 [[6. 5. 2. 3. 2.]
 [5. 6. 2. 2. 3.]
 [2. 3. 6. 5. 2.]
 [3. 2. 5. 6. 3.]
 [6. 4. 3. 2. 2.]
 [2. 2. 3. 4. 6.]]

Element-wise multiplication:
 [[25. 16.  1.  4.  1.]
 [16. 25.  1.  1.  4.]
 [ 1.  4. 25. 16.  1.]
 [ 4.  1. 16. 25.  4.]
 [25.  9.  4.  1.  1.]
 [ 1.  1.  4.  9. 25.]]
```

```
Cosine Similarity Matrix:
 [[1.    0.957 0.574 0.619 0.969 0.507]
 [0.957 1.    0.532 0.536 0.923 0.554]
 [0.574 0.532 1.    0.949 0.6   0.692]
 [0.619 0.536 0.949 1.    0.626 0.805]
 [0.969 0.923 0.6   0.626 1.    0.5  ]
 [0.507 0.554 0.692 0.805 0.5   1.   ]]

Most similar customers: 1 and 5
Cosine similarity: 0.969

Dense layer output: [8.8 4.1 3.1 2.1]
Manual calculation: [8.8 4.1 3.1 2.1]
Matches: True
```



Experiment 2: Covariance, Eigenvalues and Eigenvectors Analysis
Problem Statement: A hospital lab records several correlated blood parameters per patient. Analysts must know which parameters move together and which directions carry most of the variation. A dataset of 200 patient records with haemoglobin, RBC count, glucose, cholesterol and BMI. The lab suspects redundancy among these readings. The objective is to compute covariance and correlation, then extract eigenvalues and eigenvectors to find the principal directions of variation. Use NumPy, Pandas, Matplotlib.
Tasks:
1. Load the data into Pandas and standardise each feature.
2. Compute the covariance and correlation matrices; plot them as heatmaps.
3. Compute eigenvalues and eigenvectors with NumPy; sort by eigenvalue.
4. Plot the data in 2-D with the top two eigenvectors drawn as arrows.
5. Report the percentage of variance captured by each eigenvalue.

Formulas:

- Standardization: Converts features to approximately zero mean and unit standard deviation.
- Covariance: Measures how two features vary together.
- Correlation: Measures the strength and direction of a linear relationship, from -1 to $+1$.

- Eigenvalues: Indicate how much variance is captured along principal directions.
- Eigenvectors: Define those principal directions.

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(42)
n = 200

# Generate illustrative correlated patient measurements
health = np.random.normal(0, 1, n)
noise = lambda scale: np.random.normal(0, scale, n)

df = pd.DataFrame({
   "Haemoglobin": 13.5 + 1.2 * health + noise(0.7),
   "RBC": 4.8 + 0.45 * health + noise(0.25),
   "Glucose": 100 + 8 * health + noise(12),
   "Cholesterol": 190 + 15 * health + noise(20),
   "BMI": 25 + 2.5 * health + noise(3)
})

print("First five records:\n", df.head())

# 1. Standardize every feature
X = (df - df.mean()) / df.std(ddof=1)

print("\nStandardized data:\n", X.head())

# 2. Covariance and correlation matrices
cov_matrix = X.cov()
corr_matrix = X.corr()

print("\nCovariance Matrix:\n", cov_matrix.round(3))
print("\nCorrelation Matrix:\n", corr_matrix.round(3))

# Plot covariance and correlation heatmaps
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
```

```
for ax, matrix, title in [
    (axes[0], cov_matrix, "Covariance Matrix"),
    (axes[1], corr_matrix, "Correlation Matrix")
]:
    im = ax.imshow(matrix, cmap="coolwarm", vmin=-1, vmax=1)
    ax.set_xticks(range(5), matrix.columns, rotation=45, ha="right")
    ax.set_yticks(range(5), matrix.index)
    ax.set_title(title)
    for i in range(5):
        for j in range(5):
            ax.text(j, i, f"{matrix.iloc[i, j]:.2f}",
                    ha="center", va="center")
    fig.colorbar(im, ax=ax)

plt.tight_layout()
plt.show()

# 3. Eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix.values)

# Sort from largest to smallest eigenvalue
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order]
eigenvectors = eigenvectors[:, order]

variance_percent = eigenvalues / eigenvalues.sum() * 100

print("\nEigenvalues:\n", eigenvalues)
print("\nEigenvectors (columns):\n", eigenvectors)
print("\nVariance captured by each component (%):\n",
      np.round(variance_percent, 2))

# 4. Project data into the first two principal components
scores = X.values @ eigenvectors[:, :2]

plt.figure(figsize=(8, 6))
plt.scatter(scores[:, 0], scores[:, 1], alpha=0.6)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Patient Data Projected onto First Two PCs")
```

```
plt.axhline(0, linewidth=0.7)
plt.axvline(0, linewidth=0.7)
plt.grid(True)
plt.show()

# Plot the top two eigenvectors as arrows in the first
# two standardized feature dimensions
plt.figure(figsize=(7, 6))
plt.scatter(X["Haemoglobin"], X["RBC"], alpha=0.25)

for k in range(2):
    plt.arrow(
        0, 0,
        eigenvectors[0, k] * 3,
        eigenvectors[1, k] * 3,
        head_width=0.12,
        length_includes_head=True
    )
    plt.text(
        eigenvectors[0, k] * 3.2,
        eigenvectors[1, k] * 3.2,
        f"PC{k+1}"
    )

plt.xlabel("Standardized Haemoglobin")
plt.ylabel("Standardized RBC")
plt.title("Principal Direction Arrows in Two Feature Dimensions")
plt.grid(True)
plt.show()

# 5. Scree plot
plt.figure(figsize=(7, 4))
plt.plot(range(1, 6), variance_percent, marker="o")
plt.xticks(range(1, 6))
plt.xlabel("Principal Component")
plt.ylabel("Variance Captured (%)")
plt.title("Explained Variance by Principal Component")
plt.grid(True)
plt.show()
```

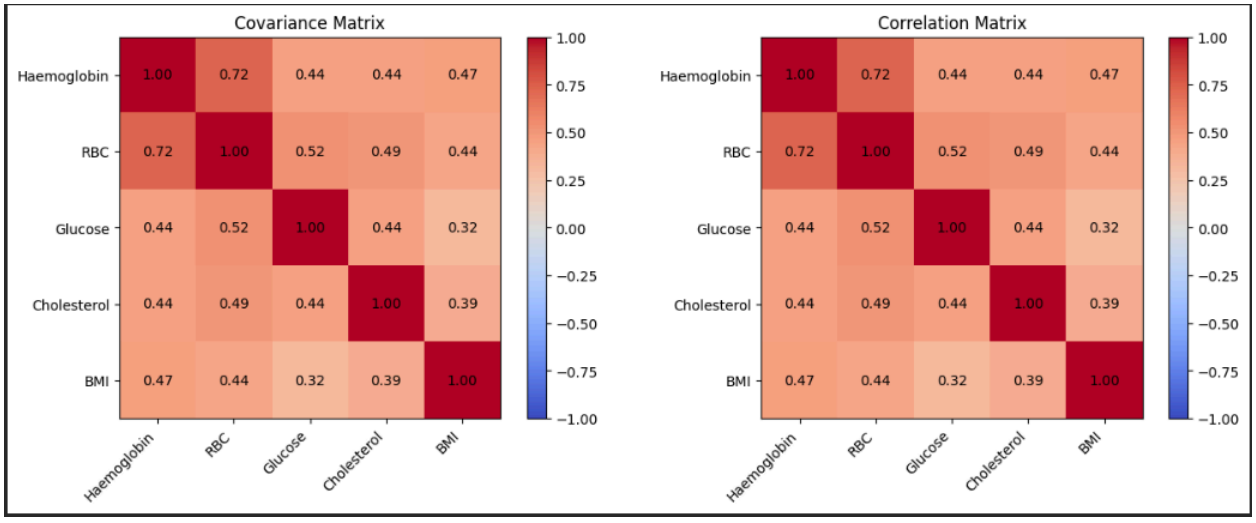
Output:

```
First five records:
    Haemoglobin       RBC     Glucose  Cholesterol        BMI
0    14.346508  4.624914  113.057577   216.216388  30.439852
1    13.726632  4.587937   87.827902   177.605141  27.428240
2    15.035362  5.092771  115.616779   201.637744  26.798112
3    16.065297  5.497109  128.451893   203.599942  26.866764
4    12.254647  4.582115  103.087992   177.797775  26.509287

Standardized data:
    Haemoglobin       RBC   Glucose  Cholesterol       BMI
0     0.610463 -0.295772  0.901068     1.040376  1.390842
1     0.157452 -0.376591 -0.811300    -0.615246  0.576064
2     1.113883  0.726796  1.074764     0.415255  0.405586
3     1.866568  1.610535  1.945899     0.499392  0.424159
4    -0.918286 -0.389317  0.224420    -0.606986  0.327445

Covariance Matrix:
              Haemoglobin    RBC  Glucose  Cholesterol    BMI
Haemoglobin         1.000  0.723    0.439        0.438  0.466
RBC                 0.723  1.000    0.524        0.493  0.435
Glucose             0.439  0.524    1.000        0.439  0.316
Cholesterol         0.438  0.493    0.439        1.000  0.390
BMI                 0.466  0.435    0.316        0.390  1.000

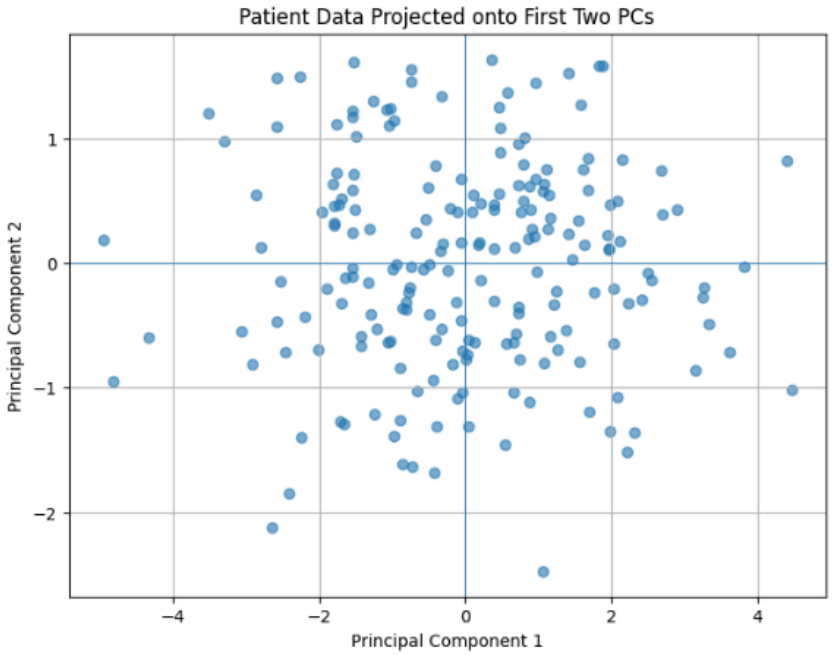
Correlation Matrix:
              Haemoglobin    RBC  Glucose  Cholesterol    BMI
Haemoglobin         1.000  0.723    0.439        0.438  0.466
RBC                 0.723  1.000    0.524        0.493  0.435
Glucose             0.439  0.524    1.000        0.439  0.316
Cholesterol         0.438  0.493    0.439        1.000  0.390
BMI                 0.466  0.435    0.316        0.390  1.000
```

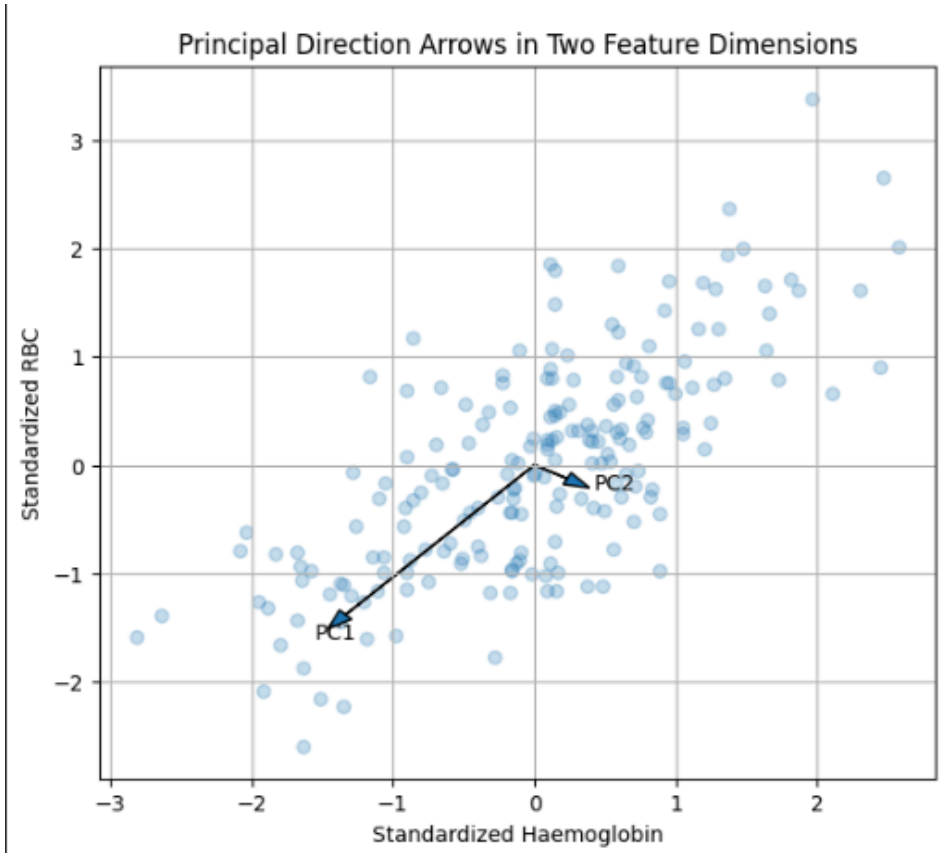


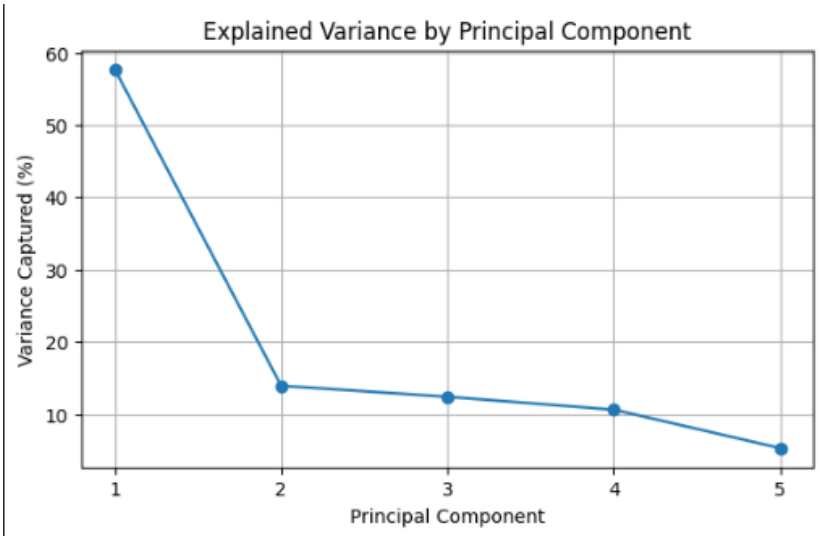

```
Eigenvalues:
 [2.88575066 0.6969751  0.62067791 0.53129892 0.26529742]

Eigenvectors (columns):
 [[-0.48630611  0.12993321  0.51340865  0.22957916 -0.65599443]
 [-0.50416499 -0.0704154   0.40345616  0.18024096  0.73864448]
 [-0.4178395  -0.60530838 -0.10706894 -0.65739709 -0.12400495]
 [-0.42346377 -0.17730158 -0.68900328  0.55697898 -0.06550943]
 [-0.39422408  0.76179062 -0.29571171 -0.41522293  0.06638471]]

Variance captured by each component (%):
 [57.72 13.94 12.41 10.63  5.31]
```


Patient Data Projected onto First Two PCs
Principal Component 2
Principal Component 1
1
0
−1
−2
−4
−2
0
2
4


Principal Direction Arrows in Two Feature Dimensions
Standardized RBC
Standardized Haemoglobin
PC1
PC2
3
2
1
0
−1
−2
−3
−2
−1
0
1
2



Experiment 3: Probability Distributions and Bayes' Theorem
Problem Statement: A diagnostic centre uses a screening test for a disease. Doctors need the probability that a patient with a positive result actually has the disease. Disease prevalence is 1%, test sensitivity 95% and specificity 90%. Patient age and test-score data follow known distributions. The objective is to model data with standard distributions and apply Bayes' theorem to a real decision. Use NumPy, SciPy, Matplotlib.
Tasks:
1. Generate and plot Bernoulli, Binomial, Poisson, Uniform and Gaussian samples with SciPy;
compare with their PDFs/PMFs.
2. Compute P(Disease | Positive) using Bayes' theorem.
3. Simulate 100,000 patients and verify the result empirically.
4. Plot how the posterior changes as prevalence varies from 0.1% to 20%.

Formulas:

Bayes' theorem is

$P(D \mid +) = \{P(+ \mid D)P(D)\} / \{P(+ \mid D)P(D)+P(+ \mid \sim D)P(\sim D)\}$

Given:

- Disease prevalence: $P(D) = 0.01$
- Sensitivity: $P(+ \mid D) = 0.95$
- Specificity: $P(- \mid \sim D) = 0.90$
- False-positive rate: $P(+ \mid \sim D) = 0.10$

Therefore,

P(D | +) = {0.95(0.01)} / {0.95(0.01)+0.10(0.99)}

$$= \frac{0.0095}{0.1085} \approx 0.0876$$

The probability that a patient actually has the disease after testing positive is approximately 8.76%.

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import bernoulli, binom, poisson, uniform, norm

np.random.seed(42)

# 1. Generate samples and compare with theoretical PMFs/PDFs

fig, axes = plt.subplots(2, 3, figsize=(14, 8))

# Bernoulli distribution
p = 0.3
sample = bernoulli.rvs(p, size=1000)
axes[0, 0].hist(sample, bins=np.arange(-0.5, 2, 1),
            density=True, rwidth=0.8)
axes[0, 0].scatter([0, 1], bernoulli.pmf([0, 1], p),
               color="red", label="Theoretical PMF")
axes[0, 0].set_title("Bernoulli")
axes[0, 0].legend()

# Binomial distribution
n_trials, p_bin = 10, 0.4
sample = binom.rvs(n_trials, p_bin, size=1000)
k = np.arange(n_trials + 1)
axes[0, 1].hist(sample, bins=np.arange(-0.5, 11.5, 1),
            density=True, rwidth=0.8)
axes[0, 1].plot(k, binom.pmf(k, n_trials, p_bin),
            "ro-", label="Theoretical PMF")
axes[0, 1].set_title("Binomial")
axes[0, 1].legend()
```

```
# Poisson distribution
lam = 4
sample = poisson.rvs(lam, size=1000)
k = np.arange(0, 13)
axes[0, 2].hist(sample, bins=np.arange(-0.5, 13.5, 1),
            density=True, rwidth=0.8)
axes[0, 2].plot(k, poisson.pmf(k, lam),
            "ro-", label="Theoretical PMF")
axes[0, 2].set_title("Poisson")
axes[0, 2].legend()

# Uniform distribution
a, b = 0, 10
sample = uniform.rvs(loc=a, scale=b-a, size=1000)
axes[1, 0].hist(sample, bins=25, density=True, alpha=0.6)
x = np.linspace(-1, 11, 300)
axes[1, 0].plot(x, uniform.pdf(x, loc=a, scale=b-a),
            "r-", label="Theoretical PDF")
axes[1, 0].set_title("Uniform")
axes[1, 0].legend()

# Gaussian (normal) distribution
mu, sigma = 50, 10
sample = norm.rvs(loc=mu, scale=sigma, size=1000)
axes[1, 1].hist(sample, bins=30, density=True, alpha=0.6)
x = np.linspace(15, 85, 300)
axes[1, 1].plot(x, norm.pdf(x, loc=mu, scale=sigma),
            "r-", label="Theoretical PDF")
axes[1, 1].set_title("Gaussian / Normal")
axes[1, 1].legend()

axes[1, 2].axis("off")
plt.tight_layout()
plt.show()

# 2. Bayes' theorem
prevalence = 0.01
sensitivity = 0.95
specificity = 0.90
```

```
false_positive_rate = 1 - specificity

p_positive = (
    sensitivity * prevalence
    + false_positive_rate * (1 - prevalence)
)

posterior = sensitivity * prevalence / p_positive

print("Probability of a positive test:", p_positive)
print("P(Disease | Positive):", posterior)
print("Percentage:", posterior * 100, "%")

# 3. Simulate 100,000 patients
N = 100_000
disease = np.random.random(N) < prevalence

test_positive = np.zeros(N, dtype=bool)

# Diseased patients test positive with sensitivity
test_positive[disease] = (
    np.random.random(disease.sum()) < sensitivity
)

# Non-diseased patients test positive at the false-positive rate
healthy = ~disease
test_positive[healthy] = (
    np.random.random(healthy.sum()) < false_positive_rate
)

positive_count = test_positive.sum()
true_positive_count = (disease & test_positive).sum()

empirical = true_positive_count / positive_count

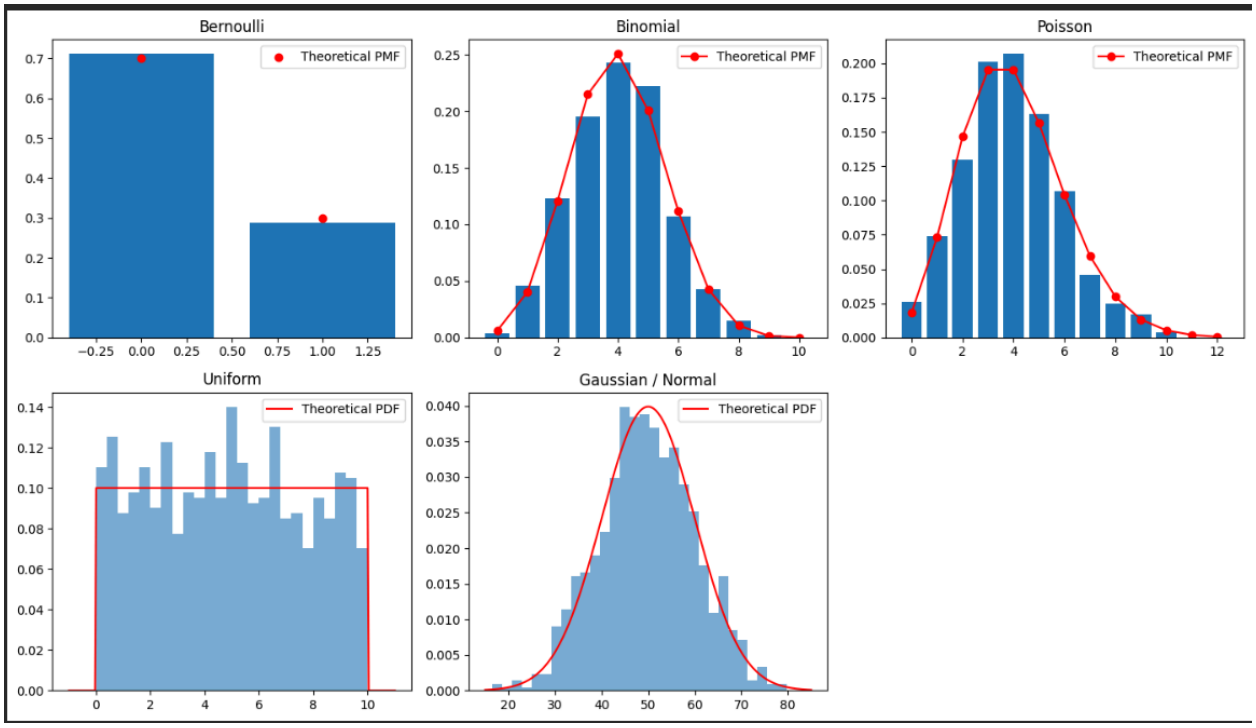
print("\nNumber of positive tests:", positive_count)
print("True positives:", true_positive_count)
print("Empirical P(Disease | Positive):", empirical)
print("Empirical percentage:", empirical * 100, "%")
```

```
# 4. Posterior probability as prevalence varies
prevalences = np.linspace(0.001, 0.20, 300)

posteriors = (
    sensitivity * prevalences
    / (
        sensitivity * prevalences
        + false_positive_rate * (1 - prevalences)
    )
)

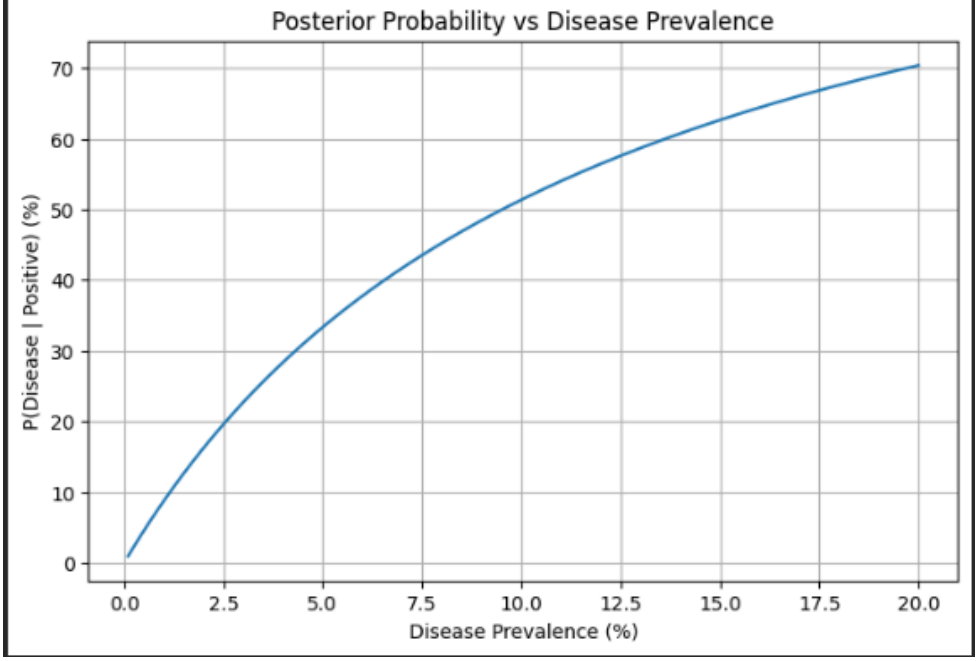
plt.figure(figsize=(8, 5))
plt.plot(prevalences * 100, posteriors * 100)
plt.xlabel("Disease Prevalence (%)")
plt.ylabel("P(Disease | Positive) (%)")
plt.title("Posterior Probability vs Disease Prevalence")
plt.grid(True)
plt.show()
```

Output:



```
Probability of a positive test: 0.10849999999999997
P(Disease | Positive): 0.08755760368663597
Percentage: 8.755760368663596 %

Number of positive tests: 10932
True positives: 964
Empirical P(Disease | Positive): 0.08818148554701793
Empirical percentage: 8.818148554701793 %
```



Experiment 4: Entropy, Cross-Entropy and KL Divergence

Problem Statement: A weather-forecasting model outputs probabilities for Sunny, Cloudy and Rainy. The team must measure how far its predictions are from what actually happened. Two competing forecast models are compared against 30 days of observed weather distributions for a city. The objective is to compute entropy, cross-entropy and KL divergence, and relate them to classification loss. Use NumPy, SciPy, Matplotlib.

Tasks:

1. Compute the entropy of the observed weather distribution.

2. Compute cross-entropy and KL divergence of each model against the true distribution.

3. Verify that cross-entropy = entropy + KL divergence.

4. Show that KL divergence is asymmetric by computing it both ways.

5. Plot binary cross-entropy loss against predicted probability.

Formulas:

Suppose the actual weather distribution over Sunny, Cloudy and Rainy is

$$P = [0.5,\ 0.3,\ 0.2]$$

The two models predict:

$$Q_1 = [0.6,\ 0.25,\ 0.15]$$

$$Q_2 = [0.2,\ 0.3,\ 0.5]$$

Entropy measures uncertainty in the true distribution:

$$H(P) = -\sum_i P_i \log_2 P_i$$

Cross-entropy measures the cost of predicting Q when the actual distribution is P:

$$H(P, Q) = -\sum_i P_i \log_2 Q_i$$

The relationship is

H(P,Q) = H(P) + D_{KL} (P || |Q)

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import rel_entr

# True observed weather distribution
P = np.array([0.5, 0.3, 0.2])

# Predictions of two weather models
Q1 = np.array([0.6, 0.25, 0.15])
Q2 = np.array([0.2, 0.3, 0.5])

weather = ["Sunny", "Cloudy", "Rainy"]

# 1. Entropy of the observed distribution
entropy = -np.sum(P * np.log2(P))
print("Entropy of true distribution:", entropy, "bits")
```

```
# 2. Cross-entropy and KL divergence
def cross_entropy(p, q):
    return -np.sum(p * np.log2(q))

def kl_divergence(p, q):
    return np.sum(rel_entr(p, q)) / np.log(2)

for name, Q in [("Model 1", Q1), ("Model 2", Q2)]:
    ce = cross_entropy(P, Q)
    kl = kl_divergence(P, Q)

    print(f"\n{name}")
    print("Cross-entropy:", ce, "bits")
    print("KL divergence:", kl, "bits")
    print("Entropy + KL:", entropy + kl)

    # 3. Verify the identity
    print("Identity verified:", np.isclose(ce, entropy + kl))

# 4. KL divergence is asymmetric
kl_pq = kl_divergence(P, Q1)
kl_qp = kl_divergence(Q1, P)

print("\nKL(P || Q1):", kl_pq)
print("KL(Q1 || P):", kl_qp)
print("Are they equal?", np.isclose(kl_pq, kl_qp))

# Plot actual distribution and both model predictions
x = np.arange(len(weather))
width = 0.25

plt.figure(figsize=(8, 5))
plt.bar(x - width, P, width, label="Actual")
plt.bar(x, Q1, width, label="Model 1")
plt.bar(x + width, Q2, width, label="Model 2")
plt.xticks(x, weather)
plt.ylabel("Probability")
plt.title("Observed Weather vs Model Predictions")
plt.legend()
```

```
plt.tight_layout()
plt.show()

# 5. Binary cross-entropy loss against predicted probability
y_true = 1
epsilon = 1e-10
predicted = np.linspace(0.001, 0.999, 500)

loss_positive = -(
    y_true * np.log(predicted)
    + (1 - y_true) * np.log(1 - predicted)
)

# Also show loss when the actual class is zero
y_zero = 0
loss_negative = -(
    y_zero * np.log(predicted)
    + (1 - y_zero) * np.log(1 - predicted)
)

plt.figure(figsize=(8, 5))
plt.plot(predicted, loss_positive, label="Actual class = 1")
plt.plot(predicted, loss_negative, label="Actual class = 0")
plt.xlabel("Predicted probability of class 1")
plt.ylabel("Binary cross-entropy loss (nats)")
plt.title("Binary Cross-Entropy Loss")
plt.ylim(0, 5)
plt.legend()
plt.grid(True)
plt.show()
```

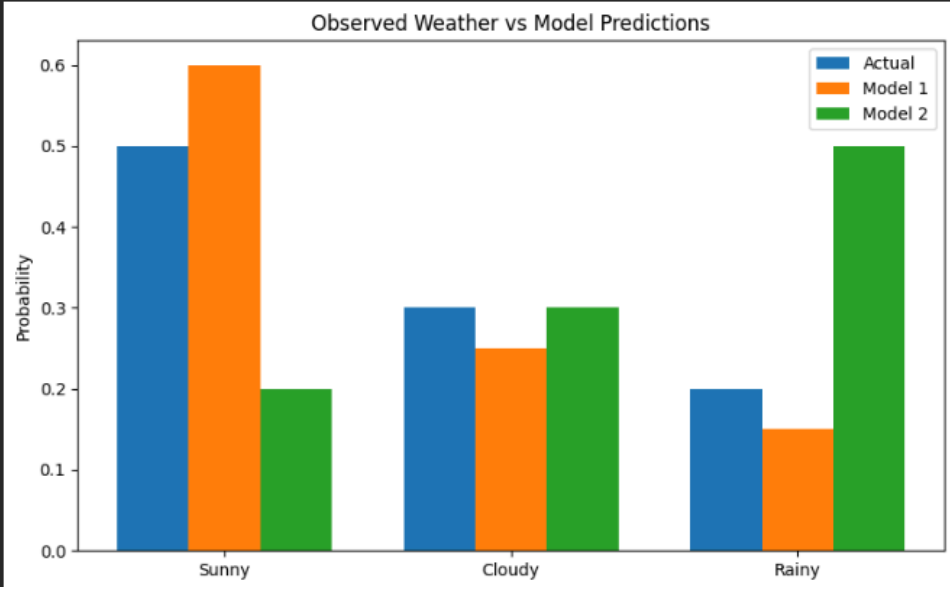
Output:

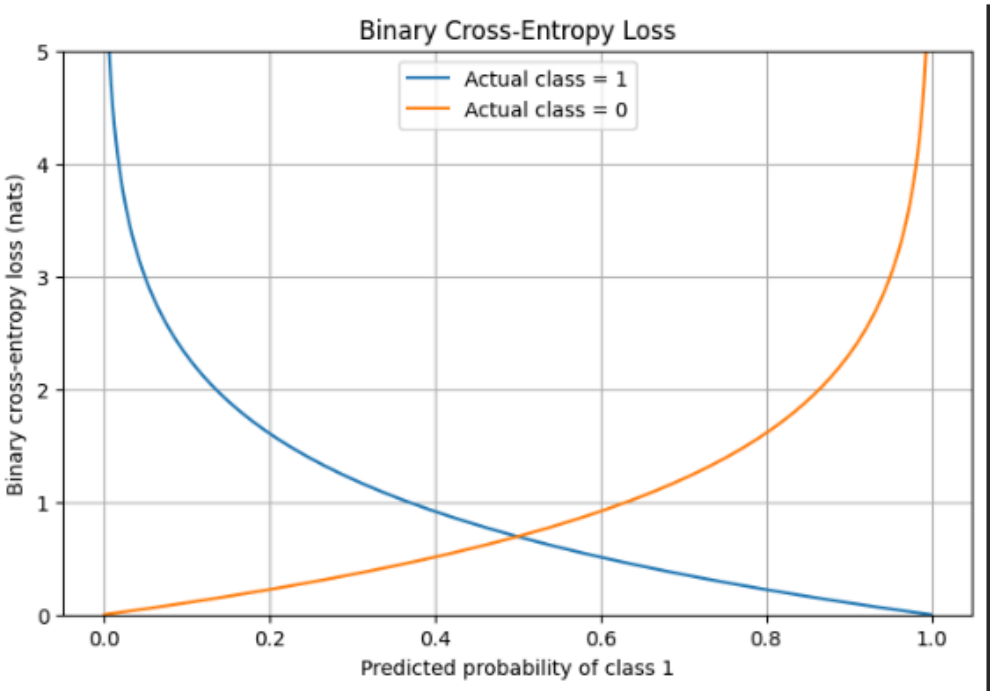
```
Entropy of true distribution: 1.4854752972273344 bits

Model 1
Cross-entropy: 1.5158759159163444 bits
KL divergence: 0.030400618689010037 bits
Entropy + KL: 1.5158759159163444
Identity verified: True

Model 2
Cross-entropy: 1.8820537256935428 bits
KL divergence: 0.3965784284662088 bits
Entropy + KL: 1.8820537256935432
Identity verified: True

KL(P || Q1): 0.030400618689010037
KL(Q1 || P): 0.029806417150001225
Are they equal? False
```





Experiment 5: Bias–Variance, Overfitting and Underfitting Analysis
Problem Statement: A real-estate start-up predicts house prices from floor area. One analyst's model is too simple; another memorises the training data. 150 noisy samples of house area versus price, with a hidden non-linear relationship. The objective is to demonstrate underfitting, good fit and overfitting, and estimate bias and variance. Use Scikit-learn, NumPy, Matplotlib.
Tasks:
1. Fit polynomial regression of degree 1, 3, 9 and 15 with Scikit-learn.
2. Plot each fitted curve over the data.
3. Plot training and validation error against model complexity.
4. Estimate bias2 and variance through repeated resampling.
5. Select the best degree and justify it.

Formulas:

The bias - variance trade-off explains why a model that is too simple or too complex may perform poorly on unseen data.

- Underfitting: A model is too simple to capture the underlying relationship. It has a high bias.
- Good fit: A model captures the main relationship and generalizes well.
- Overfitting: A model learns noise in the training data and performs poorly on new data. It often has high variance.

A common decomposition of expected squared prediction error is

$$\mathbb{E}\left[(Y - \hat{f}(X))^2\right] = \text{Bias}(\hat{f}(X))^2 + \text{Var}(\hat{f}(X)) + \sigma^2$$

Here, σ^2 represents irreducible noise under the usual assumptions.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import mean_squared_error

np.random.seed(42)

# Generate 150 noisy house-area and price samples
n = 150
area = np.sort(np.random.uniform(500, 3000, n))

# Hidden nonlinear relationship
def true_price(x):
    return (
        50
        + 0.08 * x
        + 0.000015 * x**2
        - 0.000000004 * x**3
    )

price = true_price(area) + np.random.normal(0, 25, n)

X = area.reshape(-1, 1)
y = price

# Split into training and validation data
X_train, X_val, y_train, y_val = train_test_split(
    X, y, test_size=0.25, random_state=42
)

degrees = [1, 3, 9, 15]
train_errors = []
val_errors = []
models = {}

# 1. Fit polynomial regression models
for degree in degrees:
    model = make_pipeline(
        PolynomialFeatures(degree=degree, include_bias=False),
        LinearRegression()
    )

    model.fit(X_train, y_train)
```

```
    models[degree] = model

    train_pred = model.predict(X_train)
    val_pred = model.predict(X_val)

    train_mse = mean_squared_error(y_train, train_pred)
    val_mse = mean_squared_error(y_val, val_pred)

    train_errors.append(train_mse)
    val_errors.append(val_mse)

    print(f"\nDegree {degree}")
    print("Training MSE:", round(train_mse, 2))
    print("Validation MSE:", round(val_mse, 2))

# 2. Plot each fitted curve
x_curve = np.linspace(500, 3000, 400).reshape(-1, 1)

fig, axes = plt.subplots(2, 2, figsize=(12, 9))

for ax, degree in zip(axes.flat, degrees):
    ax.scatter(X_train[:, 0], y_train, s=18,
               alpha=0.6, label="Training data")
    ax.scatter(X_val[:, 0], y_val, s=18,
               alpha=0.6, label="Validation data")

    ax.plot(
        x_curve[:, 0],
        models[degree].predict(x_curve),
        linewidth=2,
        label=f"Degree {degree}"
    )

    ax.set_title(f"Polynomial Degree {degree}")
    ax.set_xlabel("House Area")
    ax.set_ylabel("House Price")
    ax.legend()

plt.tight_layout()
plt.show()
```

```
# 3. Training and validation error vs model complexity
plt.figure(figsize=(8, 5))
plt.plot(degrees, train_errors, marker="o",
        label="Training MSE")
plt.plot(degrees, val_errors, marker="s",
        label="Validation MSE")
plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Model Complexity vs Error")
plt.xticks(degrees)
plt.legend()
plt.grid(True)
plt.show()

# 4. Estimate bias squared and variance through resampling
# Use many training samples drawn from the same underlying
# nonlinear relationship. Evaluate predictions at fixed points.
rng = np.random.default_rng(123)

X_eval = np.linspace(500, 3000, 100).reshape(-1, 1)
y_true_eval = true_price(X_eval[:, 0])

n_resamples = 100
prediction_results = {degree: [] for degree in degrees}

for repeat in range(n_resamples):
    X_boot = rng.uniform(500, 3000, 100).reshape(-1, 1)
    y_boot = (
        true_price(X_boot[:, 0])
        + rng.normal(0, 25, len(X_boot))
    )

    for degree in degrees:
        model = make_pipeline(
            PolynomialFeatures(degree=degree, include_bias=False),
            LinearRegression()
        )
        model.fit(X_boot, y_boot)
        predictions = model.predict(X_eval)
```

```
        prediction_results[degree].append(predictions)

bias_squared = []
variance = []

for degree in degrees:
    predictions = np.array(prediction_results[degree])

    mean_prediction = predictions.mean(axis=0)

    bias2 = np.mean((mean_prediction - y_true_eval) ** 2)
    var = np.mean(np.var(predictions, axis=0))

    bias_squared.append(bias2)
    variance.append(var)

    print(f"\nDegree {degree}")
    print("Estimated Bias^2:", round(bias2, 3))
    print("Estimated Variance:", round(var, 3))

# Plot estimated bias squared and variance
plt.figure(figsize=(8, 5))
plt.plot(degrees, bias_squared, marker="o",
         label="Bias squared")
plt.plot(degrees, variance, marker="s",
         label="Variance")
plt.xlabel("Polynomial Degree")
plt.ylabel("Estimated Error Component")
plt.title("Bias–Variance Trade-off")
plt.xticks(degrees)
plt.legend()
plt.grid(True)
plt.show()

# 5. Select degree with the lowest validation error
best_index = np.argmin(val_errors)
best_degree = degrees[best_index]

print("\nBest polynomial degree:", best_degree)
print("Lowest validation MSE:", round(val_errors[best_index], 2))
```

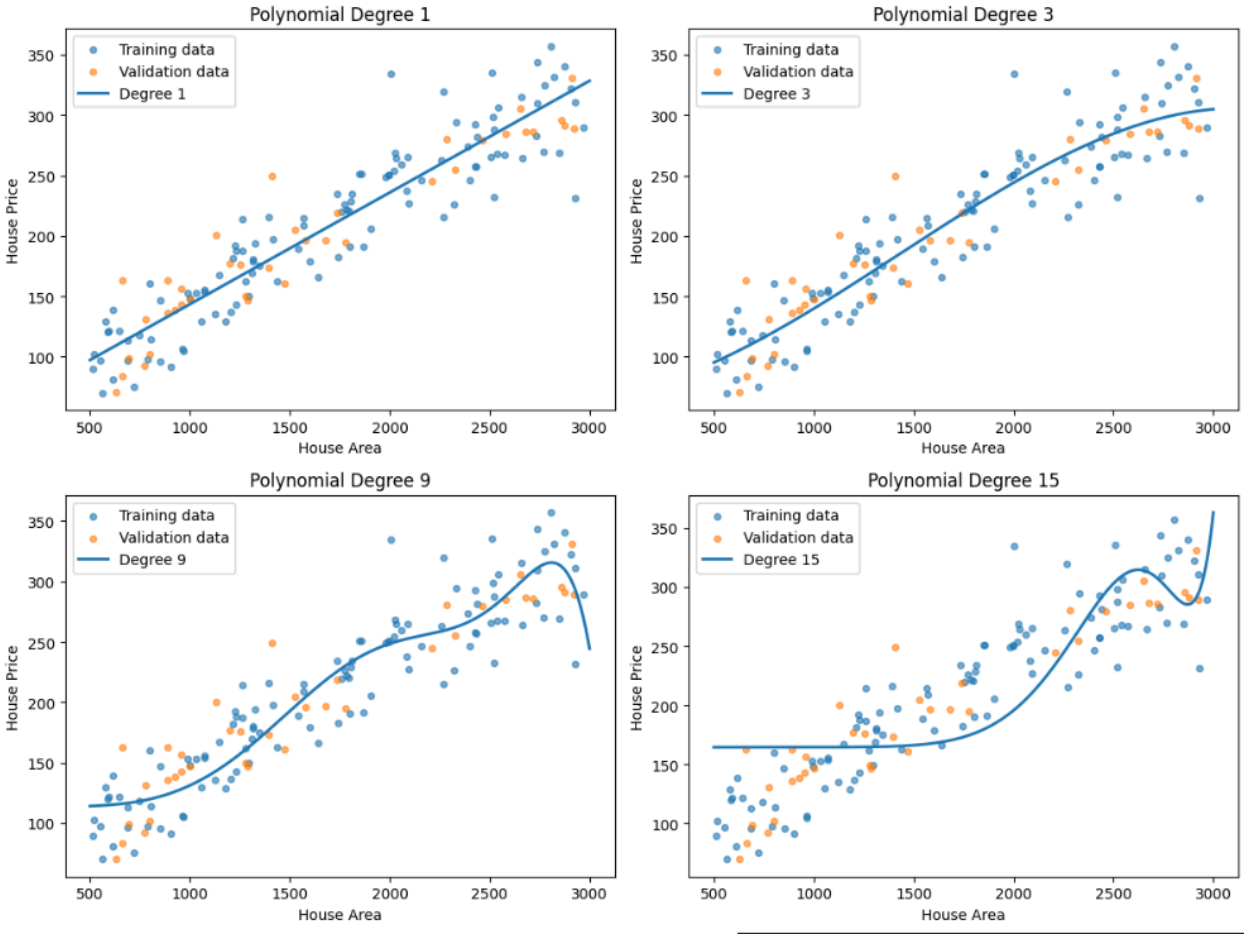
Output:

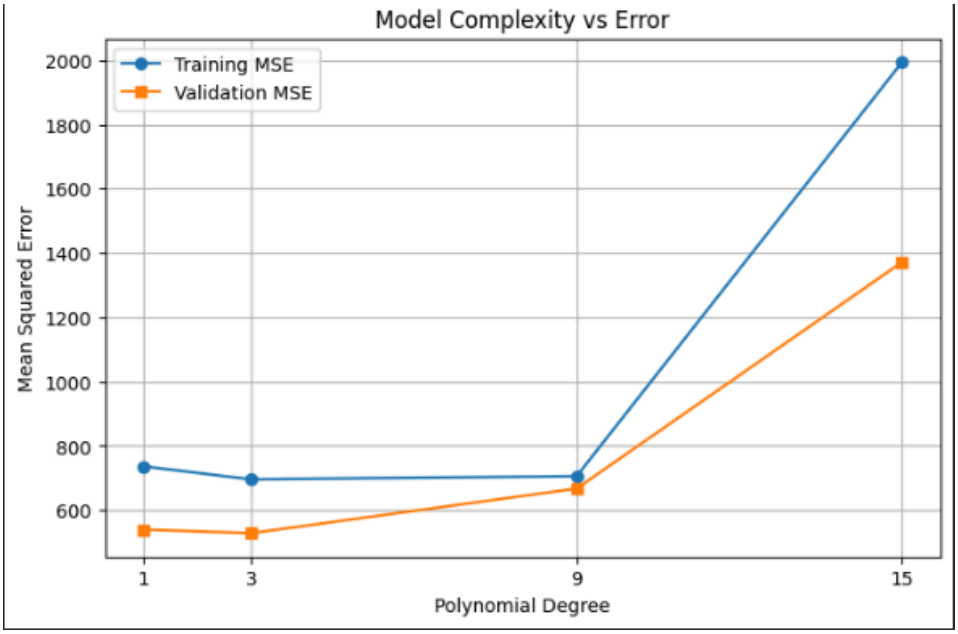
```
Degree 1
Training MSE: 736.96
Validation MSE: 540.12

Degree 3
Training MSE: 696.12
Validation MSE: 528.98

Degree 9
Training MSE: 706.29
Validation MSE: 667.65

Degree 15
Training MSE: 1993.94
Validation MSE: 1372.06
```






```
Degree 1
Estimated Bias^2: 9.723
Estimated Variance: 12.119

Degree 3
Estimated Bias^2: 0.276
Estimated Variance: 25.171

Degree 9
Estimated Bias^2: 72.4
Estimated Variance: 50.307

Degree 15
Estimated Bias^2: 1099.822
Estimated Variance: 142.697
```

